

TF-IDF, step by step — a worked example from our corpus

Study note for Group 12 (oral-defense preparation). Not a project deliverable: TF-IDF mechanics are class-covered theory, so this lives outside the notebook and report. Every number below was recomputed from the fitted vectorizers in `tm_tests_12_v2.ipynb` (scikit-learn `TfidfVectorizer`, default settings, fit on the 6,679-tweet training split only).

The example tweet

Raw	WWE stock price target cut to \$57 from \$79 at Benchmark
Cleaned	wwe stock price target cut num num benchmark

The cleaner lowercased the text, dropped stop words (*to, from, at*) and normalised both dollar amounts to the placeholder num — which is why num appears twice.

The three steps

`TfidfVectorizer` builds each tweet's vector in three steps:

$$w_{t,d} = \underbrace{\text{tf}_{t,d}}_{\text{step 1}} \times \underbrace{\text{idf}_t}_{\text{step 2}}, \quad \text{then } \underbrace{v_d \leftarrow v_d / \|v_d\|_2}_{\text{step 3}}$$

The vocabulary itself is fixed first: every term that appears in at least 2 training tweets (`min_df=2`) gets a column — 4,600 columns for unigrams, 9,104 when bigrams are added. Everything is learned from the **training split only**; validation and test tweets are only ever *transformed* with these frozen statistics (no leakage).

Step 1 — term frequency (tf)

The raw count of the term in *this* tweet. This is exactly the counts column of the notebook table: every word occurs once, num twice.

Step 2 — inverse document frequency (idf)

A rarity weight per vocabulary term, learned from the corpus. With smoothing (the sklearn default), where $N = 6679$ training tweets and df_t = number of training tweets containing term t :

$$\text{idf}_t = \left(\ln \frac{1 + N}{1 + \text{df}_t} \right) + 1$$

Worked for *wwe*, which occurs in 7 training tweets:

$$\text{idf}_{wwe} = \left(\ln \frac{6680}{8} \right) + 1 = \ln(835) + 1 = 6.727 + 1 = 7.727$$

All seven unigrams of our tweet:

term	tf	df (tweets containing it)	idf	tf · idf
benchmark	1	10	7.409	7.409
cut	1	144	4.830	4.830
num	2	1,944	2.234	4.468
price	1	288	4.140	4.140
stock	1	911	2.991	2.991
target	1	166	4.689	4.689
wwe	1	7	7.727	7.727

This is the re-weighting that makes TF-IDF useful: *wwe* and *stock* both occur once in the tweet, but *wwe* is ~130× rarer in the corpus, so its raw weight is 2.6× larger. Note also *num*: its tf of 2 only partially compensates its very low idf (it appears in 29% of all tweets), so it lands mid-table.

Step 3 – L2 normalisation (what “the norm” is)

The **L2 norm** (Euclidean norm) of a vector is its geometric length:

$$\|v\|_2 = \sqrt{\sum_i v_i^2}$$

For our tweet’s raw tf·idf vector (7 non-zero entries):

$$\|v\|_2 = \sqrt{7.409^2 + 4.830^2 + 4.468^2 + 4.140^2 + 2.991^2 + 4.689^2 + 7.727^2} = \sqrt{205.96} = 14.352$$

Every entry is then divided by this length, so the final vector has length 1:

$$w_{wwe} = \frac{7.727}{14.352} = 0.538 \quad w_{stock} = \frac{2.991}{14.352} = 0.208 \quad \dots$$

— exactly the `tfidf` column in the notebook. Why normalise? Two reasons. First, **length invariance**: without it, a 30-token tweet would have systematically larger feature values than a 6-token tweet, and the classifier would partly learn tweet length instead of content. Second, **geometry**: for unit vectors the dot product *is* the cosine similarity, which is the natural notion of document closeness for linear models and KNN.

The bigram variant

`TfidfVectorizer(ngram_range=(1, 2))` runs the identical machinery over a vocabulary of unigrams **and** adjacent word pairs (9,104 columns). Two things happen to our tweet:

1. The unigrams’ tf and idf do not change at all. Their numerators (tf · idf) are identical to the table above. What changes is step 3: the tweet now has 14 active features instead of 7, so the norm grows —

$$\|v\|_2 = \sqrt{205.96 + 297.70} = \sqrt{503.66} = 22.443$$

— and every unigram weight is divided by this larger number. That is why each unigram shrinks by the *same* factor $14.352/22.443 = 0.639$ (check: $0.516 \times 0.639 = 0.330$, $0.337 \times 0.639 = 0.215$, ...). Pure renormalisation; the relative ordering among unigrams is untouched.

2. The bigrams enter as ordinary vocabulary entries with their own df and idf — and because word pairs are much rarer than single words, their idfs are high:

bigram	df	idf	tf · idf	final weight
wwe stock	2	8.708	8.708	0.388
num benchmark	4	8.197	8.197	0.365
target cut	27	6.475	6.475	0.288
cut num	30	6.373	6.373	0.284
stock price	83	5.376	5.376	0.240
price target	110	5.097	5.097	0.227
num num	284	4.154	4.154	0.185

wwe stock (2 occurrences in the whole corpus) becomes the heaviest feature of the entire row. More importantly for this project, **target cut** — one of the mirror-verb bigrams the EDA identified as separating Bearish from Bullish (*target cut* vs *target raised*) — enters with weight 0.288, heavier than any common unigram. This is the mechanism by which the bigram variant hands the classifier the EDA’s phrase-level sentiment signal with above-average weight instead of diluting it.

What the classifier receives

One row of 9,104 numbers, 14 of them non-zero, each attached to a named n-gram. A linear model (e.g. Logistic Regression) learns one coefficient per column; its prediction for this tweet is driven by the active features — and a defensible story like “*target cut* with weight 0.29 pushed the tweet toward Bearish” can be read directly off coefficients × weights. That interpretability is what the dense representations (word2vec, transformer embeddings) trade away.